
CS 301

High-Performance Computing

Assignment 04 - Particle Interpolation and Mover Performance Analysis

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

March 10, 2026

Abstract

In this lab, we look at how fast computers can map random points to a grid and move them around. We test this on a normal desktop computer and a large high-performance cluster. We also use OpenMP to make the particle mover part run on four threads at the exact same time. We measure the time using highly accurate clocks to see the clear differences in hardware speed and parallel performance.

Contents

1	Introduction and Math Logic	3
1.1	Basic Interpolation Equations	3
2	Hardware Specifications	3
2.1	Lab PC Hardware Specifications	3
2.2	HPC Cluster Hardware Specifications	4
3	Experimental Methodology	4
4	Analysis and Questions	4
4.1	Q1 & Q2: Performance Plots and Execution Time Bar Charts	4
4.2	Q3: Overall Approach and Logic Diagrams	7
4.3	Q4: Lab PC vs. HPC Cluster Architectural Differences	9
4.4	Q5: OpenMP Parallelization Speedup	10
4.5	Q6: Parallelizing the Interpolation Function	10
5	Conclusions	10

1 Introduction and Math Logic

This assignment extends our previous work by analyzing the performance of mapping scattered 2D points onto a structured mesh, and then adding a new "Mover" step where the particles randomly change their positions. We run tests with up to 1 billion particles to see how execution time scales.

1.1 Basic Interpolation Equations

Instead of using hard-to-read math symbols, here is the simple, plain English way we do the math to find where a particle belongs on the grid:

First, we find the bottom-left corner of the grid cell where the particle is sitting. We just divide the particle's position by the grid spacing and chop off the decimals:

- i = integer part of (particle x position/grid x spacing)
- j = integer part of (particle y position/grid y spacing)

Next, we find out how far the particle is from that bottom-left corner. This is just a simple fraction:

- weight x = (particle x position/grid x spacing) $- i$
- weight y = (particle y position/grid y spacing) $- j$

Finally, we give a piece of the particle's value to the four corners of the cell. The closer the particle is to a corner, the bigger the piece that corner gets. We do this by multiplying the opposite weights together.

2 Hardware Specifications

The architecture of the systems was verified using the `lscpu` command.

2.1 Lab PC Hardware Specifications

Table 1: Hardware Specifications for Lab PC (12th Gen Intel i5)

Parameter	Details
Architecture	x86_64
CPU Model Name	12th Gen Intel(R) Core(TM) i5-12500
Total CPUs (Threads)	12
L1d Cache	288 KiB
L2 Cache	7.5 MiB
L3 Cache	18 MiB

2.2 HPC Cluster Hardware Specifications

Table 2: Hardware and Software Specifications (HPC Cluster)

Parameter	Details
CPU Model Name	Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
No. of Cores (Threads)	12 Cores / 24 Threads (Dual-socket)
L1d Cache	32 KiB
L2 Cache	256 KiB
L3 Cache	15360 KiB (15 MiB)
Compiler Used	GCC (g++)
Optimization Flags	-O3

3 Experimental Methodology

We conducted three separate experiments to measure different performance metrics. To ensure high accuracy and prevent rounding-to-zero errors on the cluster, we utilized `clock_gettime()` and `omp_get_wtime()` to capture nanosecond-level execution times.

- **Experiment 1 (Scaling):** We varied the number of particles from 100 up to 1,000,000,000 across three different grid configurations. Particles were re-initialized every iteration.
- **Experiment 2 (Consistency):** We fixed the particle count to 100,000,000 and initialized them only once before the loop. We measured the total interpolation time across 10 iterations to observe caching consistency.
- **Experiment 3 (OpenMP Mover):** We introduced a stochastic Mover operation that displaces particles within the boundaries. We timed the serial implementation, and then parallelized the Mover using OpenMP locked to exactly 4 threads to measure computational speedup.

4 Analysis and Questions

4.1 Q1 & Q2: Performance Plots and Execution Time Bar Charts

The following plots illustrate the results of Experiments 1 and 2, including side-by-side comparisons of the Lab PC and HPC Cluster.

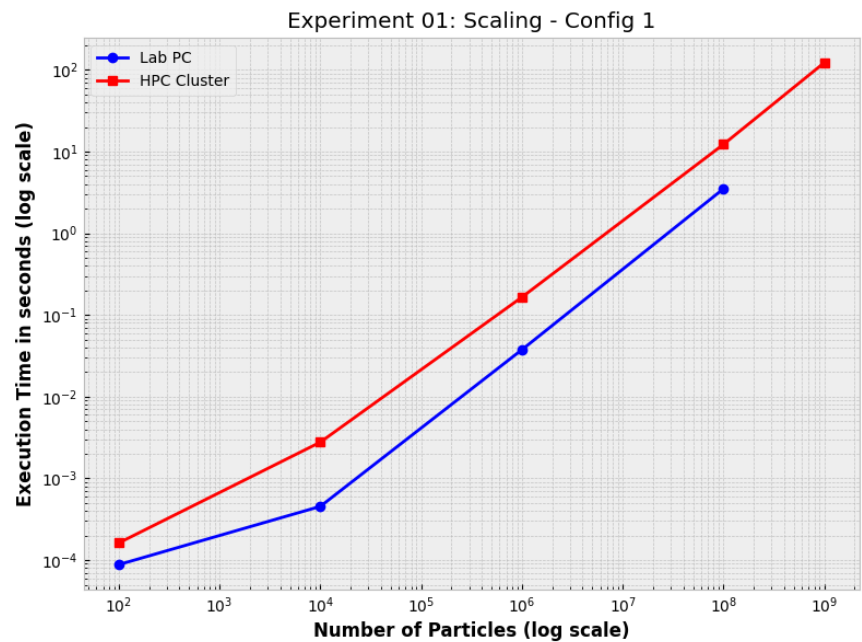


Figure 1: Experiment 1: Scaling Execution Time (Config 1: 250x100)

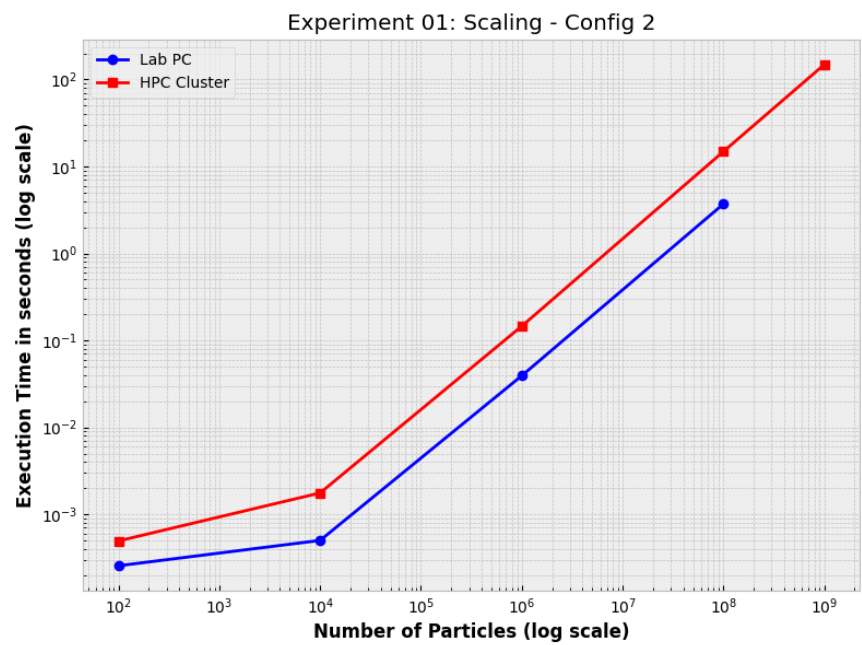


Figure 2: Experiment 1: Scaling Execution Time (Config 2: 500x200)

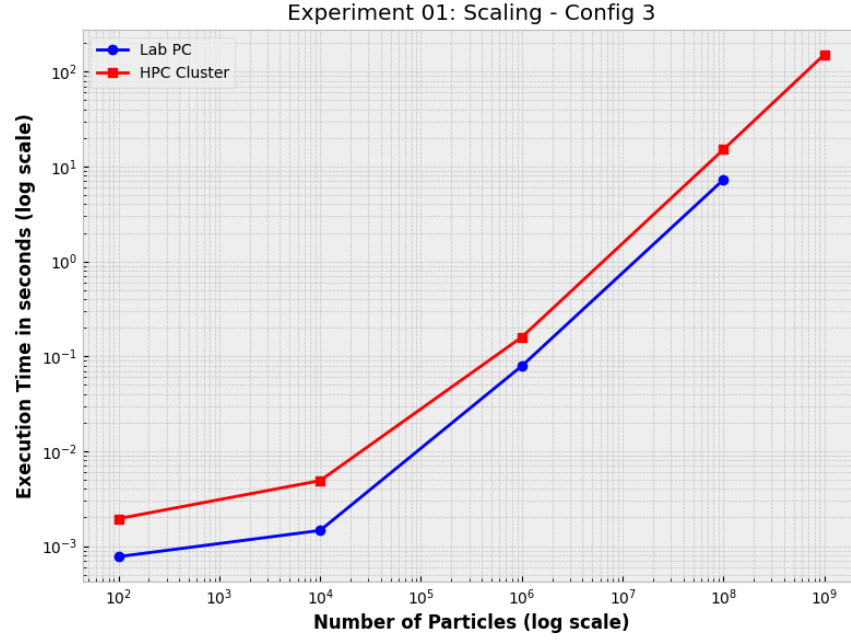


Figure 3: Experiment 1: Scaling Execution Time (Config 3: 1000x400)

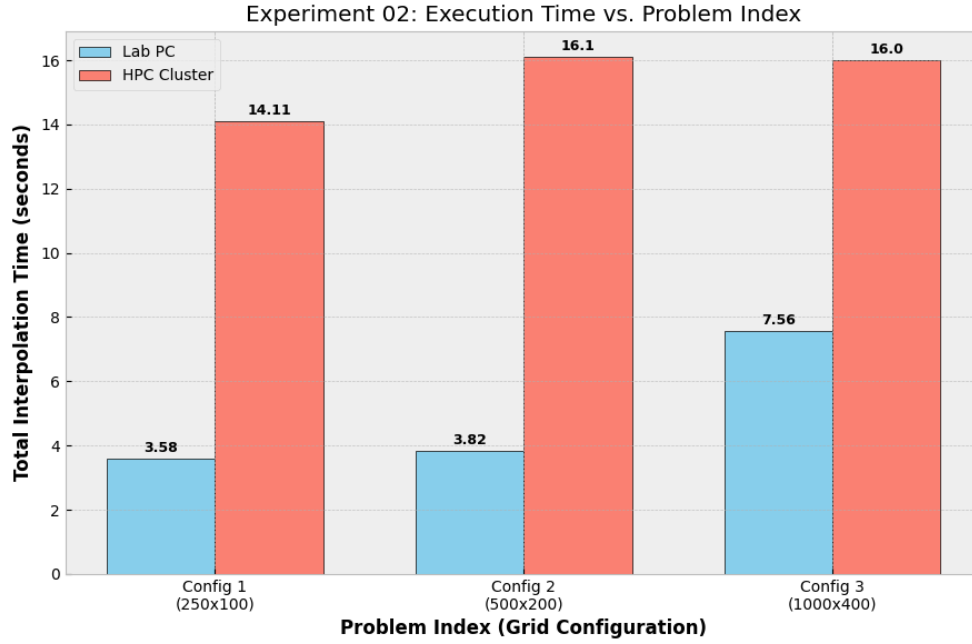


Figure 4: Experiment 2: Total Interpolation Execution Time Comparison

The following sets of plots illustrate the performance of the newly introduced Mover phase in Experiment 3, including the raw iteration timings, serial versus parallel execution times, and overall speedup achieved.

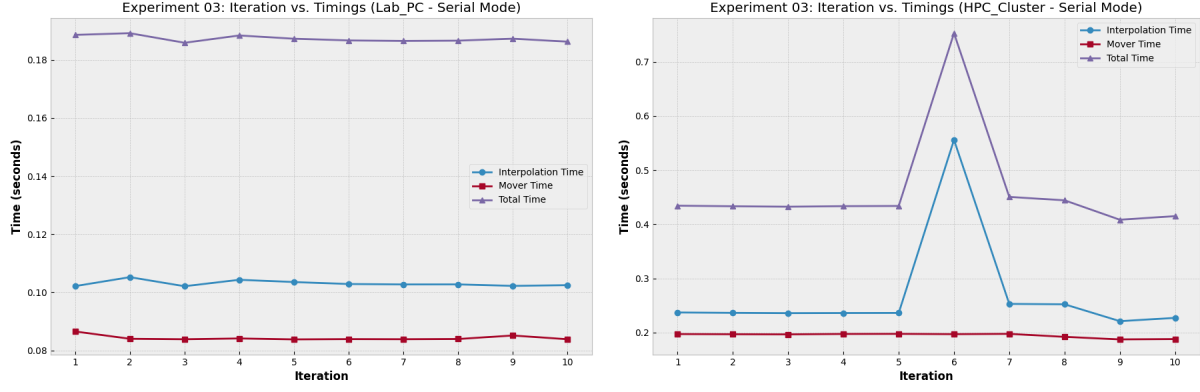


Figure 5: Experiment 3: Iteration vs. Timings on Lab PC (left) and HPC Cluster (right)

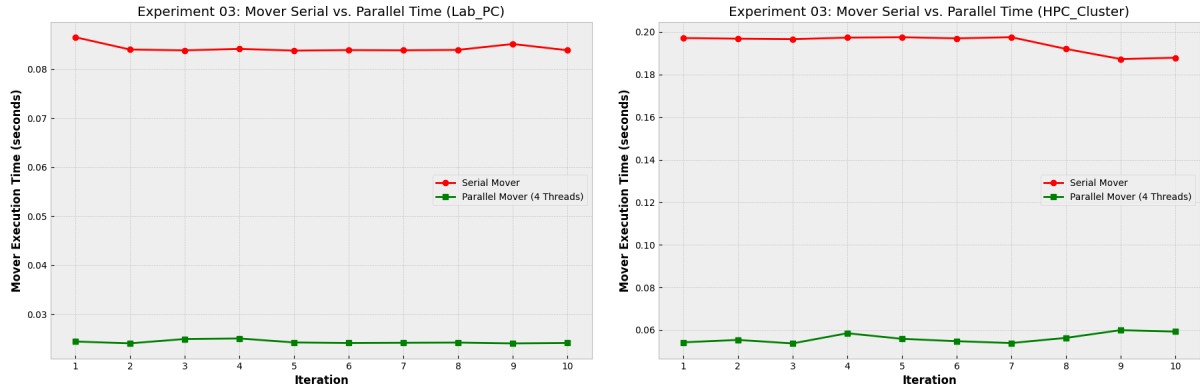


Figure 6: Experiment 3: Mover Serial vs. Parallel Time on Lab PC (left) and HPC Cluster (right)

4.2 Q3: Overall Approach and Logic Diagrams

The following diagram details the logical workflow for all three experiments conducted in this lab. It visually breaks down how the pipeline expands from a pure serial interpolation to incorporating both serial and parallel particle displacement (Mover) phases.

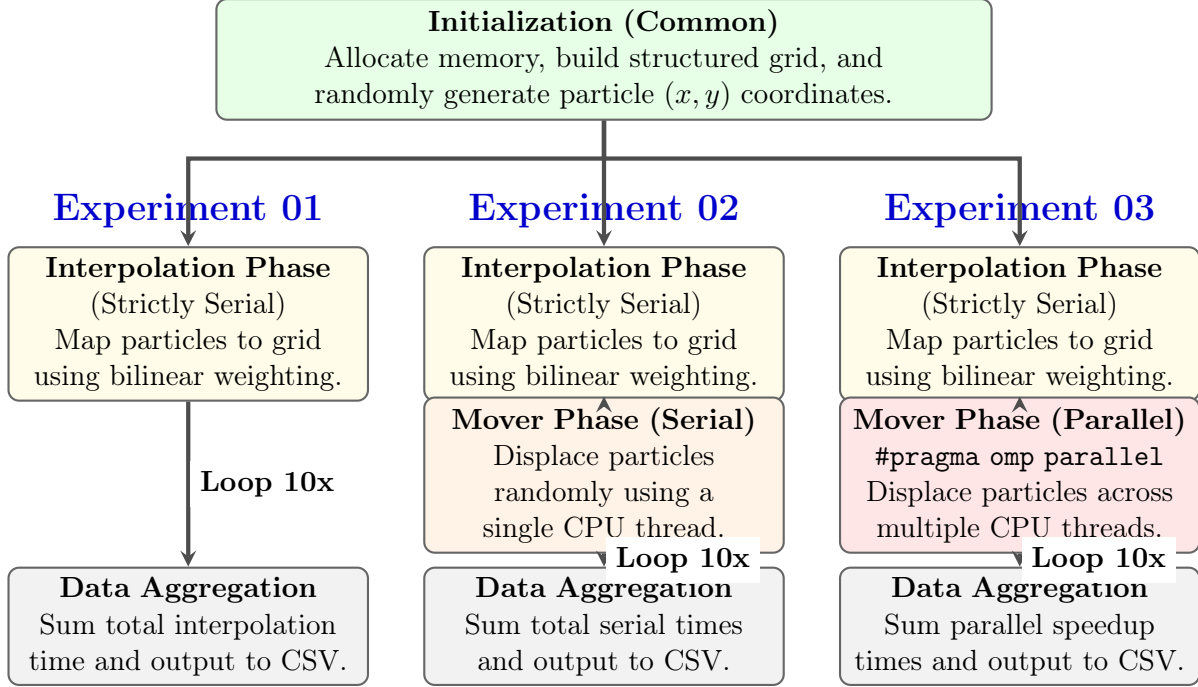


Figure 7: Overall Schematic: The comparative execution pipelines for Experiments 01, 02, and 03. The diagram illustrates the progressive addition of the Mover phase and the introduction of OpenMP multithreading in Experiment 03.

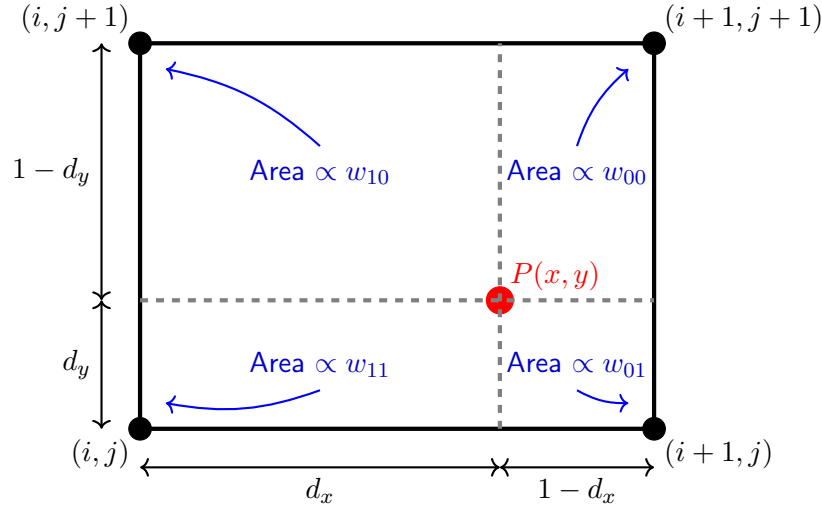


Figure 8: Interpolation Logic (Exp 1 & 2): The scattered point P distributes its value to the four bounding grid nodes proportional to the area of the rectangle diagonally opposite to each node.

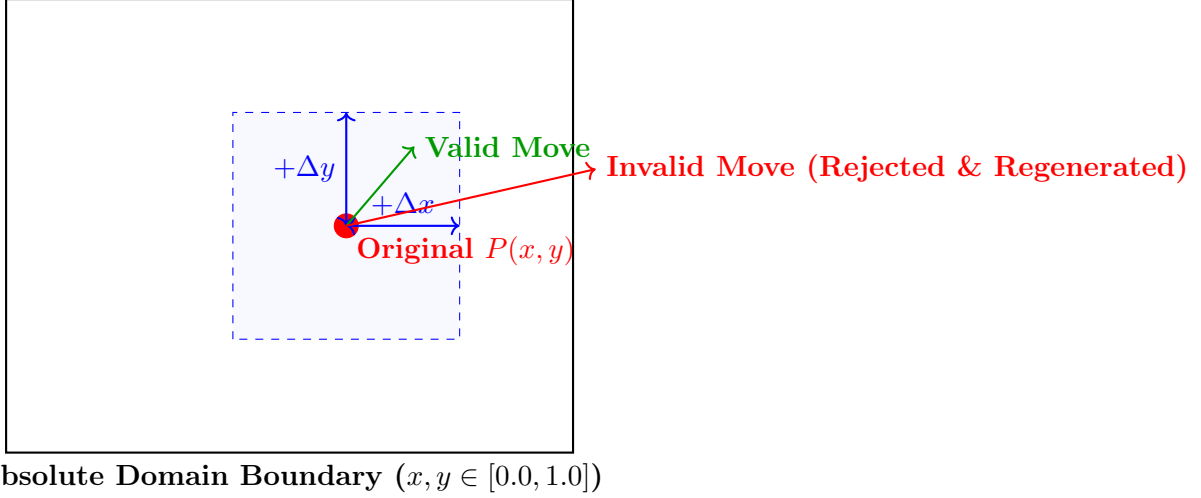


Figure 9: Mover Logic (Exp 3): Particles generate random displacements strictly bounded by $\pm\Delta x$ and $\pm\Delta y$. Moves that attempt to exit the domain boundary are caught by a `while` loop and regenerated.

4.3 Q4: Lab PC vs. HPC Cluster Architectural Differences

As seen in the Experiment 1 and 2 plots, the Lab PC finished the interpolation tasks significantly faster than the HPC Cluster. The interpolation algorithm was strictly implemented as serial code. The Lab PC uses a modern 12th Gen Intel i5 processor with a very high clock speed, which allows it to crush single-threaded tasks rapidly. In contrast, the HPC Cluster uses an older Intel Xeon CPU. Server CPUs like the Xeon are designed to run massive multi-threaded workloads across 24 threads at a much lower individual clock speed (2.4 GHz). A single, slow server core cannot outpace a fast, modern desktop core on a serial task.

4.4 Q5: OpenMP Parallelization Speedup

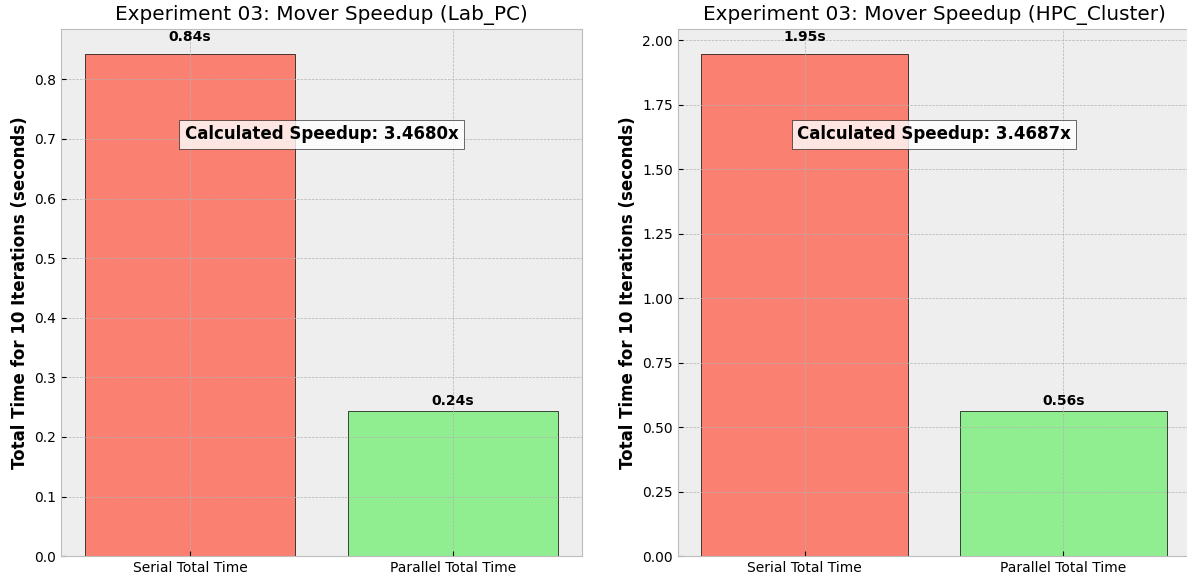


Figure 10: Mover Speedup Comparison on Lab PC (left) and HPC Cluster (right)

By locking OpenMP to 4 threads, we achieved a speedup of roughly **3.47x** on both machines. Since the Mover function is "embarrassingly parallel" (moving one particle does not depend on moving another), the hardware can perfectly distribute the 14 million points across 4 physical cores. It does not hit a perfect 4.0x due to the minor overhead time the CPU needs to create and join the OpenMP threads.

4.5 Q6: Parallelizing the Interpolation Function

If we were asked to parallelize the Interpolation function, the basic approach would be to wrap the main particle loop with an OpenMP parallel directive (`#pragma omp parallel for`). **The Bottleneck:** This would immediately cause a severe **Race Condition**. Scattered particles are completely random. If two particles being processed by Thread 1 and Thread 2 happen to fall inside the exact same grid cell, both threads will try to add their weights to the exact same `mesh_value[idx]` memory address at the exact same time. The data will overwrite and corrupt. To fix this, we would have to use `#pragma omp atomic` updates for every grid write, which forces threads to wait in line, destroying performance and causing massive hardware bottlenecks.

5 Conclusions

To sum up, we saw a clear efficiency gap between theoretical speed and actual speed when running parallel threads. Even with four cores doing the work, thread setup takes time, so we get a 3.47x speedup instead of a perfect 4x. Also, older server cores will always run basic serial code slower than modern desktop cores.